

Manual do Usuário

Risco de Fogo Previsto em Python — INPE_FireRiskModel v2.2

Versão 2.2 · 5 de agosto de 2026

Substitui: rf_previsto_1-5dias_2023.sh e rf_previsto_1-2_semanas_2024.sh (bash + NCL)

Programa Queimadas — <http://www.inpe.br/queimadas/>

Sumário

1. Visão geral
2. Requisitos
3. Instalação
4. Uso
5. Script genérico para qualquer horizonte (rf_previsto.py)
6. Preparo dos dados, RF observado e figuras (prepara_gfs, prepara_imerg, prepara_era5, rf_observado, rf_figura)
7. FWI — Canadian Fire Weather Index System
8. Saídas
9. Logs e monitoramento
10. Solução de problemas
11. Testes de validação
12. Segurança
13. Suporte

1. Visão geral

Este pacote calcula o Risco de Fogo (RF) previsto em resolução de 1 km para a América do Sul, combinando a precipitação observada do IMERG (últimos 119 dias), as previsões do GFS (precipitação, temperatura e umidade relativa a 2 m), o mapa de vegetação (MapBiomass/IGBP) e a topografia (GTOPO30). São três programas:

- **rf_previsto_1_5dias.py** — gera 19 previsões, de +6 h até +4 dias 18 UTC, a cada 6 horas (produto RF_PREV).
- **rf_previsto_1_2_semanas.py** — gera 2 previsões, para +7 e +14 dias às 18 UTC (produto RF_PREV_SEMANAL).
- **rf_previsto.py** — script genérico: gera o RF para qualquer horizonte de previsão e diferentes fontes de dados (GFS, Eta, BESM) informados na linha de comando (seção 5).
- **rf_observado.py** — RF observado dos últimos dias, semanas ou meses, com IMERG + ERA5, incluindo médias do período e mensais (seção 6.4).
- **rf_figura.py** — figuras PNG dos campos de RF na paleta oficial da operação (seção 6.5).
- **fwi_core.py / fwi_observado.py** — motor do FWI (Canadian Fire Weather Index System) e o FWI observado diário (seção 7).

Ambos usam o módulo comum **rf_core.py** e produzem, para cada horário de previsão, um NetCDF (RF.PREV.YYYYMMDDHH.nc) e um GeoTIFF (RF.PREV.YYYYMMDDHH.tif), além dos produtos derivados (links D1–D5, cópias T0–T4/T7/T14 e fogograma).

2. Requisitos

2.1 Software

Python 3.9 ou superior e as bibliotecas:

```
pip install numpy xarray netCDF4 rasterio pyyaml
```

Para rodar o teste de validação, instale também o scipy:

```
pip install scipy
```

Não são mais necessários: NCL, cdo, gdal (binário), GNU parallel nem o ambiente conda ncl_stable.

2.2 Dados de entrada

Dado	Local esperado	Observação
Precipitação IMERG (diária)	data/output/2.2/Precipitation-2_2/AAAA/MM/INPE_FireRiskModel_2.2_Precipitation_AAAAMMD D.nc	119 dias anteriores à execução; variável prec
Previsões do GFS	data/output/2.2/GFS/netcdf/AAAAMMD00/	GFS.PREV.PREC.* e GFS.PREV.TEMP2m.RH2m.*
Mapa de vegetação 1 km	data/input/Veg_Map_2020/Merge_MapBiomass_V5_IGBP_C6_<ano>.nc	Band1, classes 0–7, sul→norte; anos ≥ 2020 usam 2019
Topografia 1 km	data/input/topografia/GeoTOPOAmericaSulCentral_V3.nc	Band1 (metros), mesma grade da vegetação

Todos os caminhos partem do diretório base. O padrão embutido (em `rf_config.py`) é `/p/projetos/grpeta/Team/jorge.gomes/risco-fogo-python`; a precedência é `--base (CLI) > base do --config > variável de ambiente RF_BASE > padrão embutido`. Exceção: os dois scripts legados (`rf_previsto_1_5dias.py` e `rf_previsto_1_2_semanas.py`) mantêm a constante `BASE = /home/queimadas/INPE_FireRiskModel` da produção, editável no topo de cada um.

3. Instalação

Copie os arquivos para o diretório de scripts do modelo (os testes são opcionais):

```
rf_core.py
rf_fontes.py
rf_config.py
config_exemplo.yaml
ativa_riscofogo.sh
rf_previsto_1_5dias.py
rf_previsto_1_2_semanas.py
rf_previsto.py
prepara_gfs.py
prepara_imerp.py
prepara_era5.py
rf_observado.py
rf_figura.py
fwi_core.py
fwi_observado.py
era5_tempo.py
config_besm.yaml
teste_rf.py
teste_rf_previsto.py
teste_rf_multifonte.py
teste_prepara_gfs.py
teste_prepara_imerp.py
teste_prepara_era5.py
teste_rf_observado.py
```

Os scripts devem ficar no mesmo diretório (os orquestradores fazem `import rf_core`). Se desejar, torne-os executáveis:

```
chmod +x rf_previsto_1_5dias.py rf_previsto_1_2_semanas.py
```

3.1 Ambiente Python no cluster (`ativa_riscofogo.sh`)

Em clusters, o erro mais comum é instalar/rodar com interpretadores diferentes (o `pip` do sistema instala num lugar, o `python3` do ambiente lê de outro). O `ativa_riscofogo.sh` resolve isso — use SEMPRE com `source`, a cada login (ou adicione ao `~/.bashrc`):

```
source ativa_riscofogo.sh
```

Ele carrega o módulo Anaconda do sistema (se existir; nome ajustável em `MODULO_ANACONDA`), ativa o env `conda riscofogo` — localizado por nome ou por caminho (envs fora do diretório padrão aparecem sem nome no `conda env list`) —, detecta env `conda` sem Python próprio (caindo para o `venv` `~/envs/riscofogo` e imprimindo o `conda install -p ...` para populá-lo) e confere versão e bibliotecas ao final. Regra de ouro: dentro de qualquer ambiente, instale com `python3 -m pip install ...`, nunca com `pip` solto.

4. Uso

4.1 Execução operacional (data de hoje)

```
python3 rf_previsto_1_5dias.py
python3 rf_previsto_1_2_semanas.py
```

O comportamento é o mesmo dos shell scripts originais: as datas são calculadas a partir do dia corrente, as saídas vão para os mesmos diretórios e, ao final, os arquivos são publicados nos servidores.

4.2 Opções de linha de comando

Opção	Efeito	Padrão
--data-final YYYYMMDD	Executa como se "hoje" fosse a data informada (reprocessamento)	data corrente
--jobs N	Número de previsões calculadas em paralelo	20 (diário) / 2 (semanal)
--sem-envio	Não publica nos servidores (geoserver/lftp/scp) — útil para testes	envio ativado

Exemplos:

```
# Reprocessar 1º/08/2026 sem publicar, usando 8 processos
python3 rf_previsto_1_5dias.py --data-final 20260801 --jobs 8 --sem-envio

# Rodada semanal de teste
python3 rf_previsto_1_2_semanas.py --sem-envio
```

4.3 Agendamento (cron)

Substitua as entradas existentes que chamavam os shell scripts, por exemplo:

```
30 6 * * * cd /home/queimadas/INPE_FireRiskModel/scr/risco_fogo/RF_Previsto \
&& /usr/bin/python3 rf_previsto_1_5dias.py \
>> /home/queimadas/INPE_FireRiskModel/log/cron.rf_prev.log 2>&1
```

5. Script genérico para qualquer horizonte (rf_previsto.py)

O rf_previsto.py generaliza os dois produtos: os horizontes de previsão são informados na linha de comando, contados a partir das 00 UTC da data do modelo. Um horizonte pode ser escrito como Nh (horas), Nd (dias), Nm (meses de calendário), combinações como 4d18h ou 1m15d, ou como data/hora absoluta YYYYMMDDHH.

5.1 Exemplos

```
# Um único horizonte: 36 horas
python3 rf_previsto.py --horizontes 36h

# Lista de horizontes: 1, 3, 7 e 14 dias às 18 UTC
python3 rf_previsto.py --horizontes 18h,2d18h,6d18h,13d18h

# Intervalo (equivalente ao produto diário de 1 a 5 dias)
python3 rf_previsto.py --de 6h --ate 4d18h --passo 6h
```

```
# Equivalente ao produto semanal (7 e 14 dias)
python3 rf_previsto.py --horizontes 7d18h,14d18h --rb-max 0.8 --fallback-gfs

# Data/hora absoluta com reprocessamento
python3 rf_previsto.py --data-final 20260801 --horizontes 2026080618

# Eta: previsões mensais de 1 a 13 meses
python3 rf_previsto.py --fonte eta --de 1m --ate 13m --passo 1m

# BESM (acúmulos de 12 h agregados automaticamente): 6 meses
python3 rf_previsto.py --fonte besm --horizontes 6m
```

5.2 Opções

Opção	Efeito	Padrão
--horizontes LISTA	Horizontes separados por vírgula (Nh, Nd, Nm, combinações ou YYYYMMDDHH)	—
--fonte NOME	Fonte de previsão: gfs, eta, besm ou outra definida via JSON (seção 5.4); padrao (ou legado/nenhuma) força a composição legada do GFS mesmo com fonte no YAML	composição legada (GFS)
--config-fontes ARQ	Arquivo JSON que ajusta/acrescenta fontes (nomes de arquivos, variáveis, frequência)	—
--de / --ate / --passo	Intervalo de horizontes relativo (pode ser combinado com --horizontes)	passo 6h
--rb-max X	Risco básico máximo (o produto semanal usa 0.8)	0.9
--produto NOME	Subdiretório de saída em data/output/2.2/	RF_PREV_CUSTOM
--base DIR	Diretório base do modelo (precedência: --base > base do --config > variável RF_BASE > padrão embutido; produção: /home/queimadas/INPE_FireRiskModel)	/p/projetos/grpeta/Team/jorge.gomes/risco-fogo-python
--fallback-gfs	Se o GFS do horário exato não existir, usa o horário anterior do mesmo dia (generaliza a cópia 12 UTC → 18 UTC do produto semanal)	desativado
--sem-tif	Gera apenas os NetCDF, sem GeoTIFF	TIF ativado
--fogograma	Gera também um único NetCDF com todos os horizontes	desativado
--sem-vegetacao	Sensibilidade: desliga o efeito da vegetação; o mapa não é lido (classe uniforme --classe-veg, saída na grade da precipitação, sem máscara d'água); sufixo _SEMVEG no produto	desativado
--sem-topografia	Sensibilidade: desliga o Fator Topográfico (FTOP=1; arquivo de topografia dispensado); sufixo _SEMTPO no produto	desativado
--classe-veg N	Classe usada com --sem-vegetacao	4 (A=2,4; PSE_max=75)
--media-mensal / --media / --maximo	Agregações da rodada: média por mês-calendário, média de toda a rodada, ou máximo (seção 5.7)	desativado
--so-agrega	Só agrega os arquivos já existentes (não recalcula)	desativado
--data-final / --jobs N	Como nos demais scripts; aceita também "hoje" (= data do sistema, útil no YAML)	hoje / 4

As saídas seguem o mesmo formato dos demais produtos (RF.PREV.YYYYMMDDHH.nc e .tif), gravadas em data/output/2.2/<produto>/netcdf|tif/<modelo>/. O script genérico não gera links D1–D5, cópias T7/T14 nem faz envio a servidores — para os produtos operacionais, use os scripts dedicados.

5.3 Fontes de dados e horizontes longos (--fonte)

O alcance depende da fonte de previsão escolhida com --fonte:

Fonte	Alcance	Frequência padrão dos acúmulos
gfs	~16 dias	1 dia
eta	até 13 meses	1 dia
besm	até 13 meses	1 dia (arquivo único por variável)

Sem --fonte, o script usa a composição original dos produtos operacionais: 119 dias de IMERG observado + o acumulado do GFS do dia previsto (idêntica aos scripts dedicados, limitada ao alcance do GFS).

Com --fonte, a série diária de 120 tempos que antecede cada data prevista é montada de forma completa: IMERG observado para os dias anteriores à rodada e precipitação PREVISTA da fonte para os dias entre a rodada e a data válida. É isso que viabiliza horizontes de semanas a 13 meses (Eta/BESM) — num horizonte de 13 meses, os 120 dias da janela são inteiramente previstos. Os acúmulos da fonte podem vir em qualquer frequência (1h, 12h, 1d): são somados para o passo diário automaticamente, e campos em grade diferente da do IMERG são regradeados por interpolação bilinear. O horizonte pedido é validado contra o alcance da fonte (horizonte_max_dias).

5.4 Configuração das fontes (rf_fontes.py e --config-fontes)

Cada fonte é descrita em rf_fontes.py por: **layout** dos arquivos (por_tempo = um arquivo por horário previsto, como o GFS deste pipeline; serie = um arquivo por variável com todos os tempos da rodada, como o BESM T062), subdiretório dos dados, padrões de nome dos arquivos (com os marcadores {modelo} e {valida}), nomes das variáveis (var_prec, var_temp, var_ur), frequência dos acúmulos (freq_prec: 1h, 12h, 1d), tipo de acumulação (intervalo = acumulado do próprio intervalo; desde_inicio = acumulado desde o início da rodada), unidades (unidade_temp: K/C; unidade_ur: %/frac) e alcance (horizonte_max_dias).

A fonte **besm** já vem configurada com a **convenção real do BESM T062 (pacote besm_queimada do CPTEC)**: layout serie com tmp_prec.nc (mm/dia), tmp_t2mt.nc (K, média diária) e tmp_rsmt.nc (% , 0–100) em BESM/netcdf/<modelo>/, 396 tempos diários (rodada+1 até rodada+13 meses), grade gaussiana ~1,875° com latitude norte→sul (invertida automaticamente na leitura). Como a série começa em rodada+1, o dia da própria rodada é preenchido com o IMERG observado, se existir, ou aproximado pelo primeiro dia da série (aviso no log). Os padrões do Eta seguem PROVISÓRIOS — ajuste-os à convenção real com um JSON, sem alterar o código:

```
{
  "eta": {
    "subdir": "ETA/netcdf/{modelo}",
    "padrao_prec": "ETA.PREV.PREC.{modelo}.{valida}.nc",
    "padrao_temp_ur": "ETA.PREV.TEMP2m.RH2m.{modelo}.{valida}.nc",
    "var_prec": "prec",
    "freq_prec": "1d",
    "horizonte_max_dias": 396
  },
}
```

```
"besm": { "freq_prec": "12h" }
}
```

```
python3 rf_previsto.py --fonte eta --horizontes 13m --config-fontes
fontes.json
```

Os arquivos das fontes devem ser NetCDF em grade regular lat/lon (como os do GFS já pré-processados neste pipeline); saídas nativas (grib do Eta, espectral do BESM) precisam ser convertidas antes. Novas fontes podem ser acrescentadas no mesmo JSON — o nome da chave passa a valer em --fonte.

5.5 Arquivo de configuração YAML (--config)

O --config arquivo.yaml centraliza toda a configuração dos dados de entrada num único arquivo YAML (ou JSON), com seções opcionais — o que não for informado usa os padrões da produção, e a linha de comando sempre prevalece sobre o arquivo:

Seção	Conteúdo
base	Diretório base do modelo; os caminhos relativos são ancorados nele
caminhos	Onde estão os dados de entrada: imerg_dir, imerg_subpastas ({ano}/{mes}), imerg_padrao ({data}), mapa_vegetacao ({ano_veg}), topografia, log
fontes	Mesmas chaves do --config-fontes: ajusta ou acrescenta fontes (gfs, eta, besm, ...)
execucao	Padrões da linha de comando: fonte, horizontes ou de/ate/passo, data_final, rb_max, produto, jobs, fallback_gfs, sem_tif, fogograma, sem_vegetacao, sem_topografia, classe_veg

```
base: /home/queimadas/INPE_FireRiskModel
caminhos:
  imerg_dir: data/output/2.2/Precipitation-2_2
  imerg_padrao: "INPE_FireRiskModel_2.2_Precipitation_{data}.nc"
fontes:
  besm: { horizonte_max_dias: 396 }
execucao:
  fonte: besm
  de: 1m
  ate: 13m
  passo: 1m
  data_final: hoje          # data do sistema (tambem: auto, sistema) – ou
  "20260804"
  produto: RF_PREV_BESM
  sem_vegetacao: false     # sensibilidade (secao 5.6): true dispensa o mapa
  sem_topografia: false    # true dispensa a topografia (FTOP = 1)
  classe_veg: 4

python3 rf_previsto.py --config config.yaml          # tudo do arquivo
python3 rf_previsto.py --config config.yaml --horizontes 6m  # CLI prevalece
```

Com data_final: hoje (equivalente a omitir a chave) e horizontes (ou de/ate/passo) declarados no arquivo, o YAML define uma rodada diária operacional: cada execução usa a data corrente do sistema e produz sempre os mesmos horizontes pré-estabelecidos — ideal para agendar no cron. As chaves de sensibilidade permitem manter arquivos separados por experimento (ex.: config.yaml de referência e config_semveg.yaml com sem_vegetacao: true), sem alterar a linha de comando.

Atenção com a chave fonte: sem ela (e sem --fonte na CLI), a rodada usa a composição original dos produtos operacionais com o GFS — este é o padrão. Se o YAML declarar fonte: besm (ou outra), essa fonte vale em toda execução que não passar --fonte na linha de comando (precedência CLI > YAML). Para forçar o GFS sem editar o arquivo, use --fonte padrao (aceita também legado/nenhuma). Por isso

o config_exemplo.yaml traz a chave comentada: declare-a apenas em arquivos dedicados a uma fonte específica (ex.: config_besm.yaml).

O arquivo config_exemplo.yaml traz o modelo completo comentado; as seções são validadas na leitura (chave desconhecida gera erro com a lista das válidas). Requer PyYAML (pip install pyyaml) — arquivos .json funcionam sem ele.

5.6 Análise de sensibilidade (--sem-vegetacao / --sem-topografia)

Para medir o impacto individual de cada componente do modelo, as duas chaves podem ser desligadas de forma independente (e combinadas):

```
python3 rf_previsto.py --data-final 20260804 --horizontes 3d #
referência
python3 rf_previsto.py --data-final 20260804 --horizontes 3d --sem-topografia
python3 rf_previsto.py --data-final 20260804 --horizontes 3d --sem-vegetacao
python3 rf_previsto.py --data-final 20260804 --horizontes 3d --sem-vegetacao
--sem-topografia
```

Semântica: topografia desligada zera a correção ($FTOP=1$) e dispensa o arquivo de topografia; vegetação desligada dispensa o mapa de vegetação (o arquivo não é lido): todos os pontos recebem uma classe uniforme (--classe-veg, padrão 4) e a saída passa a usar a grade da precipitação — sem interpolação para 1 km e sem máscara d'água. Com as duas chaves ligadas, o RF roda sem nenhum arquivo estático (útil quando o mapa de vegetação e a topografia ainda não estão disponíveis); se a topografia permanecer ligada com a vegetação desligada, a elevação é regradeada automaticamente para a grade da precipitação. Os fatores de latitude e meteorológicos permanecem ativos. Proteções: o produto ganha sufixo automático (_SEMVEG/_SEMTPO), nunca sobrescrevendo a referência, e o NetCDF registra os fatores desligados nos atributos globais (fator_vegetacao/fator_topografia). As chaves também existem no YAML (sem_vegetacao, sem_topografia, classe_veg).

5.7 Risco médio: agregações da rodada (--media-mensal / --media)

Cada horizonte gera o seu RF.PREV.{data}{hora}.nc. Para obter o risco médio — típico das rodadas sazonais (Eta/BESM), em que interessa o mês e não o dia — o rf_previsto.py agrega os campos ao final:

Opção	Saída
--media-mensal	RF.PREV.MEDIA.AAAAMM.nc — uma média por mês-calendário coberto pelas previsões
--media	RF.PREV.MEDIA.{ini}-{fim}.nc — média de toda a rodada
--maximo	usa o máximo em vez da média (RF.PREV.MAXIMO.*)
--so-agrega	não recalcula nada: agrega os arquivos já existentes da rodada

Os campos são agrupados pela data válida de cada previsão, e as médias ignoram valores ausentes ponto a ponto (o número de campos usados vai no atributo global dias_agregados). As mesmas chaves existem no YAML (media_mensal, media, maximo).

Rodada sazonal do BESM (1 a 13 meses). O BESM traz previsão diária por ~13 meses (396 dias), então há duas formas de produzir "um valor por mês":

```
# (a) Instantaneo: 13 mapas, um no mesmo dia de cada mes - rapido (13
calculos)
python3 rf_previsto.py --fonte besm --de 1m --ate 13m --passo 1m \
--produto RF_PREV_BESM

# (b) Risco MEDIO de cada mes: usa todos os dias previstos (396 calculos)
```

```
python3 rf_previsto.py --fonte besm --de 1d --ate 396d --passo 1d \
  --media-mensal --media --produto RF_PREV_BESM --jobs 8 --sem-tif
```

```
# Se os diários já existirem, so as medias (nao recalcula nada):
python3 rf_previsto.py --fonte besm --de 1d --ate 396d --passo 1d \
  --media-mensal --so-agrega --produto RF_PREV_BESM
```

A forma (b) é a recomendada quando o objetivo é a climatologia mensal prevista: o mapa de cada mês passa a representar todos os dias, e não um dia específico. O arquivo `config_besm.yaml` já traz essa configuração pronta — basta `python3 rf_previsto.py --config config_besm.yaml`. As figuras dos 13 mapas saem com o `rf_figura.py` (seção 6.5).

```
python3 rf_figura.py <saida>/RF.PREV.MEDIA.2026*.nc --painel --colunas 4 \
  --titulo "BESM - Risco de Fogo medio mensal"
```

Atenção ao passo: ele tem a mesma unidade dos horizontes e precisa ser compatível com o intervalo — `--de 6h --ate 4d18h --passo 1m` não produz nada útil (o primeiro passo já ultrapassa o fim).

6. Preparo dos dados, RF observado e figuras (`prepara_gfs`, `prepara_imerg`, `prepara_era5`, `rf_observado`, `rf_figura`)

Dois scripts geram o banco de dados de entrada do RF sem depender da área de produção do Programa Queimadas: o `prepara_gfs.py` (previsões) e o `prepara_imerg.py` (precipitação observada). Fluxo completo de uma rodada:

```
python3 prepara_imerg.py --config config.yaml # 1. IMERG observado
(119 dias)
python3 prepara_gfs.py --config config.yaml # 2. previsões do GFS
python3 rf_previsto.py --de 6h --ate 4d18h --passo 6h # 3. RF
```

6.1 GFS (`prepara_gfs.py`)

O `prepara_gfs.py` baixa a previsão do GFS 0,25° da NOAA e grava os arquivos `GFS.PREV.PREC.*` (prec em mm/dia) e `GFS.PREV.TEMP2m.RH2m.*` (T2m em K, UR2m em %) na convenção da seção 2.2, prontos para o `rf_previsto.py`. Requer o `pygrib` (`python3 -m pip install pygrib`).

Importante: o serviço OpenDAP do NOMADS foi aposentado pela NOAA (Service Change Notice 25-81, efetivo em 23/02/2026). O script usa os serviços vigentes indicados no próprio aviso:

Método	Descrição
s3 (padrão)	"Fast download method": lê o índice .idx de cada horário e baixa por byte-range só as 3 mensagens GRIB necessárias, no espelho oficial da AWS (noaa-gfs-bdp-pds), ~2 MB por horário
nomads	O mesmo fast download, direto no HTTPS do NOMADS (/pub/data/nccf/com/gfs/prod/...)
filtro	Grib filter do NOMADS (recorte de variáveis/região no servidor; URL ajustável via --url-filtro)

```
python3 prepara_gfs.py # rodada de hoje 00 UTC, 16 dias
python3 prepara_gfs.py --data 20260805 --config config.yaml
python3 prepara_gfs.py --simular # só mostra o plano e a URL
python3 prepara_gfs.py --metodo nomads # se a AWS estiver bloqueada
```

Opções principais: --data/--rodada (rodada), --dias (alcance, padrão 16), --passo (validades, padrão 6 h), --dominio latS,latN,lonW,lonE, --acumulo 24h|dia, --jobs (downloads simultâneos), --base/--config (destino), --sobrescrever e --simular.

Sobre a precipitação: o APCP do GFS vem em "baldes" (6 h até +240 h; 12 h de +240 h a +384 h). O acumulado diário de cada validade soma os baldes que cobrem as 24 h anteriores, com o intervalo lido do próprio GRIB — a transição 6 h/12 h é automática, e validades sem arquivo além de +240 h (fora do passo de 12 h) são puladas com aviso.

6.2 IMERG (prepara_imerg.py)

O prepara_imerg.py baixa a precipitação diária observada do IMERG — produto GPM_3IMERGDE V07 (Early Daily, ~4 h de latência, o mesmo da operação) — do GES DISC/NASA e converte para o padrão de leitura do pipeline (INPE_FireRiskModel_2.2_Precipitation_AAAAMMDD.nc, variável prec em mm/dia, lat sul→norte, domínio recortado), no destino definido pela seção caminhos do config.yaml.

Pré-requisito (uma única vez): conta gratuita no Earthdata (urs.earthdata.nasa.gov) e um dos dois modos de autenticação:

- **Modo A — token (recomendado):** no perfil do Earthdata, aba Generate Token, copie o token e salve no servidor: `echo 'SEU_TOKEN' > ~/.edl_token && chmod 600 ~/.edl_token` (ou exporte EARTHDATA_TOKEN). Dispensa autorização de app e senha no disco.
- **Modo B — usuário/senha:** autorize o app "NASA GESDISC DATA ARCHIVE" em Applications → Authorized Apps (obrigatório) e crie o ~/.netrc (chmod 600; o login é o nome de usuário, não o e-mail):

```
machine urs.earthdata.nasa.gov login SEU_USUARIO password SUA_SENHA
```

Se a autenticação falhar, o script diz o motivo (página de LOGIN = usuário/senha errados; pedido de AUTORIZAÇÃO = app não aprovado; 401 = token inválido/expirado).

Modos de uso:

```
python3 prepara_imerg.py                # data corrente: 119 dias
até ontem
python3 prepara_imerg.py --data-final 20260804  # janela da rodada
(reprocessamento)
python3 prepara_imerg.py --inicio 20250101 --fim 20251231  # período
explícito
python3 prepara_imerg.py --produto final ...  # histórico consolidado da
NASA
python3 prepara_imerg.py --simular          # confere período e URLs
```

O script pula dias já existentes (rodar de novo completa o que faltou; --sobrescrever regrava), tenta as letras de versão da NASA automaticamente (V07D→V07A), baixa em paralelo (--jobs) e trata a transposição lon/lat e os valores inválidos do formato IMERG. Cada dia baixa ~25 MB do arquivo global e grava ~2–3 MB recortados. Produtos: early (padrão), late (~14 h) e final (pesquisa, meses de latência — ideal para períodos históricos).

6.3 ERA5 (prepara_era5.py)

O prepara_era5.py baixa a reanálise ERA5 (Copernicus/ECMWF, 0,25°) e converte para o padrão de leitura do RF: temperatura a 2 m, umidade relativa a 2 m calculada da temperatura + ponto de orvalho (fórmula de Magnus, Alduchov & Eskridge 1996) e vento a 10 m (u10/v10, para uso futuro). Para cada dia são gravados dois arquivos no destino da chave era5_dir do config.yaml:

Arquivo	Variáveis
ERA5.OBS.TEMP2m.RH2m.{data}{hora}.nc	TEMP2m (K) e RH2m (%) — lido por rf_core.ler_temp_ur, mesmo formato do GFS
ERA5.OBS.U10m.V10m.{data}{hora}.nc	U10m e V10m (m/s)

Pré-requisito (uma única vez): conta gratuita no CDS (cds.climate.copernicus.eu), aceitar no site a licença do dataset "ERA5 hourly data on single levels", instalar a API (python3 -m pip install cdsapi) e criar o ~/.cdsapirc (chmod 600):

```
url: https://cds.climate.copernicus.eu/api
key: SEU-TOKEN-PESSOAL
```

Modos de uso:

```
python3 prepara_era5.py --config config.yaml           # últimos 7 dias
disponíveis
python3 prepara_era5.py --inicio 20260601 --fim 20260731 # período
explícito
python3 prepara_era5.py --data-final 20260730 --dias 120 # janela p/
rf_observado
python3 prepara_era5.py --simular                       # só o plano (sem
baixar)
```

A ERA5 tem atraso de ~5 dias (dias recentes vêm do produto preliminar ERA5T, tratado automaticamente), por isso a janela padrão termina 6 dias atrás. As requisições ao CDS são agrupadas por mês (mais eficientes na fila do Copernicus — cada uma pode esperar alguns minutos); a hora da análise é configurável (--hora, padrão 18 UTC, a mesma dos produtos do RF), dias completos são pulados e --sobrescrever regrava.

Download incremental. O script confere o que já existe por par (dia, hora) antes de pedir qualquer coisa ao CDS: um dia só é considerado pronto quando os dois arquivos daquela hora existem (T/UR e vento). O que já está no disco não é baixado nem regravado (para refazer, use --sobrescrever), então uma execução interrompida é retomada sem custo — basta rodar de novo que ele completa apenas as lacunas. O --simular mostra exatamente esse plano, uma requisição por mês e por hora:

```
Arquivos (dia x hora): 21; 5 já existem, 16 a baixar
requisicao CDS: 2026-06 as 17 UTC, 2 dia(s): [2, 3]
requisicao CDS: 2026-06 as 18 UTC, 1 dia(s): [3]
```

Dica de volume. No modo solar, o número de horas depende da largura do domínio: o padrão (-114,95 a -30,05, que alcança o Pacífico) exige 7 horas UTC; restringindo ao Brasil (--dominio -35,7,-75,-32) caem para 4 — quase metade do volume. Confira antes se o domínio menor cobre toda a grade do IMERG, já que os produtos observados usam a grade da precipitação como referência.

6.4 Risco de Fogo observado (rf_observado.py)

O rf_observado.py calcula o RF observado de dias que já passaram — últimos dias, semanas ou meses — usando apenas observações: a série completa de 120 dias do IMERG (incluindo o próprio dia analisado) e a T2m/UR2m da ERA5 na hora da análise. É a mesma formulação do modelo (rb_max 0,9 e fatores FU/FT/FLAT/FTOP), o que permite reconstituir o histórico recente e comparar com as previsões (RF.PREV.* × RF.OBS.*).

```
python3 rf_observado.py --config config.yaml --dias 7      # últimos 7 dias
python3 rf_observado.py --config config.yaml --semanas 2   # últimas 2
semanas
python3 rf_observado.py --config config.yaml --meses 3     # últimos 3
meses-calendário
python3 rf_observado.py --config config.yaml --de 20260601 --ate 20260731
```

```
python3 rf_observado.py --config config.yaml --dias 7 --simular # confere as
entradas
python3 rf_observado.py --config config.yaml --dias 7 --sem-vegetacao --sem-
topografia
```

O fluxo completo é: prepara_imerp.py (precipitação do período + 119 dias anteriores) → prepara_era5.py (T/UR do período) → rf_observado.py. O --simular confere as entradas e aponta o que falta; dias com entradas incompletas são pulados com aviso (e o comando de preparo sugerido), sem interromper os demais. As saídas RF.OBS.{data}{hora}.nc (e .tif) vão para data/output/2.2/RF_OBS/netcdf (produto configurável via --produto, com os sufixos automáticos _SEMVEG/_SEMTOPO da análise de sensibilidade). Demais opções como no rf_previsto.py: --hora, --rb-max, --jobs, --sem-tif, --classe-veg, --data-final (aceita 'hoje'; padrão 6 dias atrás, pelo atraso da ERA5).

Um arquivo por dia + agregações. Cada dia do período gera o seu RF.OBS.{data}{hora}.nc. Para o risco médio — como nos mapas mensais do BESM — acrescente:

Opção	Saída
--media	RF.OBS.MEDIA.{ini}-{fim}.nc — média de todo o período
--media-mensal	RF.OBS.MEDIA.AAAAMM.nc — uma média por mês-calendário do período
--maximo	usa o máximo em vez da média (arquivos RF.OBS.MAXIMO.*)
--mergetime	RF.OBS.SERIE.{ini}-{fim}.nc — todos os dias num só arquivo, com eixo de tempo
--so-agrega	não recalcula nada: agrega os diários já existentes no período

As médias ignoram valores ausentes ponto a ponto (a contagem efetiva de dias usados fica no atributo global dias_agregados), e os arquivos saem no mesmo formato dos diários — podem ser abertos no QGIS/GeoServer ou passados ao rf_figura.py.

```
# Julho de 2026: 31 mapas diarios + a media do mes
python3 rf_observado.py --de 20260701 --ate 20260731 --media

# Jan-jul/2026: media de cada mes + media do periodo todo
python3 rf_observado.py --de 20260101 --ate 20260731 --media-mensal --media

# So as medias, a partir dos diarios ja calculados
python3 rf_observado.py --de 20260701 --ate 20260731 --media --so-agrega
```

6.5 Figuras dos campos de RF (rf_figura.py)

O rf_figura.py gera PNG de qualquer NetCDF do pipeline (RF.OBS.*, RF.PREV.*, médias mensais) usando a paleta oficial da operação, idêntica ao SLD do GeoServer (INPE_FireRiskModel_2.2): -999 transparente e as paradas #17b617 (Mínimo, 0,15), #79f674 (Baixo, 0,40), #ffff82 (Médio, 0,70), #ff2e00 (Alto, 0,95) e #a70000 (Crítico, 1,00), interpoladas como no type="ramp" do SLD.

```
# Um mapa (a figura vai para o lado do NetCDF, se --saida for omitido)
python3 rf_figura.py data/output/2.2/RF_OBS/netcdf/RF.OBS.MEDIA.202607.nc

# Painel com as medias mensais do ano
python3 rf_figura.py data/output/2.2/RF_OBS/netcdf/RF.OBS.MEDIA.2026*.nc \
  --painel --titulo "Risco de Fogo observado - media mensal" \
  --saida rf_obs_mensal_2026.png

# Todos os dias de julho, 7 colunas, faixas discretas
python3 rf_figura.py '.../RF.OBS.202607*18.nc' --painel --colunas 7 --classes
```

Opções: --painel (vários campos numa figura) com --colunas, --titulo, --saida (arquivo ou pasta), --dpi, --classes (uma cor por faixa, em vez da rampa — útil para leitura por classe) e --sem-mascara (não aplica a máscara de oceano, necessária apenas em campos gerados com --sem-vegetacao, que não trazem a máscara d'água). Requer matplotlib (e global-land-mask para a máscara de oceano).

6.6 Horário da ERA5: hora fixa ou hora solar local

As variáveis da ERA5 são instantâneas (o valor da hora cheia), e é isso que os índices de perigo pedem: as condições do momento mais crítico do dia — o meio da tarde no RF, o meio-dia local no FWI. Uma média diária não seria equivalente: ela rebaixa a temperatura, sobe a umidade e comprime a variabilidade, subestimando o risco de forma sistemática (num dia típico de seca no Cerrado, trocar as 18 UTC pela média do dia derruba o fator de temperatura em ~13 %).

Há, porém, um problema real com a hora fixa num domínio largo: 18 UTC é 16 h no litoral do Nordeste e 13 h no Acre. Por isso a seção era5 do config.yaml oferece os dois modos:

```
era5:
  horario: fixo          # fixo | solar
  hora: 18              # hora UTC no modo fixo (o que a operacao usa)
  hora_local: 15        # hora solar local no modo solar (12 = convencao do
FWI)
  # horas: [17, 18, 19, 20] # opcional: horas UTC explicitas
```

No modo solar, cada faixa de longitude (fusos solares de 15°) usa a hora UTC correspondente à hora local pedida, e o campo é montado faixa a faixa. Sobre o Brasil, hora_local: 15 precisa das horas 17, 18, 19 e 20 UTC; hora_local: 12 (FWI) precisa de 14 a 17 UTC. O prepara_era5.py lê a mesma seção e baixa exatamente essas horas:

```
python3 prepara_era5.py --config config.yaml --dias 30 # horas conforme o
config
python3 prepara_era5.py --dias 30 --hora 17,18,19,20 # horas explicitas
python3 prepara_era5.py --config config.yaml --simular # mostra o plano por
hora
```

Os produtos observados (rf_observado.py e fwi_observado.py) seguem a mesma configuração, com --horario e --hora-local para sobrepor pontualmente. No modo solar o produto ganha o sufixo _SOLAR e os arquivos são rotulados pela hora local (RF.OBS.2026073115.nc), de modo que as duas convenções nunca se misturam. Custo a considerar: o modo solar multiplica o volume baixado da ERA5 pelo número de horas (4x sobre o Brasil, mais em domínios que chegam ao Pacífico).

6.7 Escala da umidade relativa no Fator de Umidade (correcao_ur)

Um achado da conversão que merece atenção: o NCL original converte a UR de porcentagem para fração ($ur2m = ur2m/100$.) antes de aplicar $FU = -0,008 \cdot UR + 1,3$. Com a UR entre 0 e 1, o FU fica praticamente constante (1,292 a 1,300) — ou seja, na prática a umidade quase não modula o RF operacional, e o produto sai multiplicado por ~1,3. Os coeficientes, porém, parecem ter sido pensados para a UR em porcentagem: aí o FU iria de 1,14 (UR 20 %) a 0,58 (UR 90 %), que é o comportamento que a descrição do modelo sugere.

Como a conversão para Python reproduz o NCL fielmente, o padrão continua sendo o comportamento da operação. A chave correcao_ur permite quantificar o efeito da escolha:

Valor	UR usada no FU	FU a 40 % de UR
ncl (padrão)	fração (0–1)	1,297
decimos	décimos (0–10)	1,268

Valor	UR usada no FU	FU a 40 % de UR
percentual	porcentagem (0-100)	0,980

execucao:

```
correcao_ur: ncl          # ncl | decimos | percentual
python3 rf_observado.py --de 20260701 --ate 20260731 --correcao_ur percentual
python3 rf_previsto.py --horizontes 3d --correcao_ur percentual
```

Recomendação: qualquer valor diferente de ncl acrescenta sufixo ao produto (_URDEC, _URPER) e registra a escolha no atributo global `escala_ur_no_FU`, para nunca se misturar com a rodada de referência. Trate como análise de sensibilidade e valide com o autor do modelo antes de qualquer mudança na operação.

7. FWI — Canadian Fire Weather Index System

Além do RF do INPE, o pacote traz o FWI canadense completo, o índice único proposto na metodologia multi-horizonte. Os dois convivem: consomem os mesmos arquivos de entrada e escrevem no mesmo padrão de saída, o que permite compará-los ponto a ponto.

7.1 O motor (`fwi_core.py`)

Implementação vetorizada em numpy dos seis componentes do sistema, na ordem de cálculo:

Componente	O que representa	Memória
FFMC	umidade do combustível fino	~2/3 de dia
DMC	umidade da camada orgânica	~12 dias
DC	seca profunda	~52 dias
ISI	FFMC + vento (espalhamento inicial)	—
BUI	DMC + DC (combustível disponível)	—
FWI	ISI + BUI (índice final)	—

Também é calculado o DSR (Daily Severity Rating). As entradas seguem a convenção do sistema — condições do meio-dia local: temperatura (°C), umidade relativa (%), vento a 10 m (km/h) e precipitação acumulada nas 24 h anteriores (mm). O ajuste hemisférico é feito por faixas de latitude (tabelas de duração do dia para DMC e DC), de modo que o comportamento sazonal fica correto no Hemisfério Sul e na faixa equatorial.

Validação: o motor é comparado, no teste `fwi.py`, com a tabela de referência do sistema — incluindo o exemplo clássico de Van Wagner & Pickett ($T=17^{\circ}\text{C}$, $UR=42\%$, $\text{vento}=25\text{ km/h}$, $\text{chuva}=0$, partindo de $\text{FFMC}=85$, $\text{DMC}=6$, $\text{DC}=15$, que deve dar $\text{FFMC } 87,7 \cdot \text{DMC } 8,5 \cdot \text{DC } 19,0 \cdot \text{ISI } 10,9 \cdot \text{BUI } 8,5 \cdot \text{FWI } 10,1$) — e, quando o xclim está instalado, também contra a implementação de referência do CFFWIS, em cinco faixas de latitude e quatro meses, com diferença menor que $1\text{e-}9$.

7.2 FWI observado (`fwi_observado.py`)

Calcula o FWI diário a partir do IMERG (chuva) e da ERA5 (T, UR e vento — daí a importância do `prepara_era5.py` já baixar U10m/V10m). Diferente do RF, que é independente por dia, o FWI é sequencial: cada dia parte dos códigos de umidade do dia anterior. O script cuida disso de duas formas: rodando um período de aquecimento (spin-up, padrão 90 dias) antes do primeiro dia pedido, e permitindo salvar/retomar o estado entre execuções.


```
# Ultimos 30 dias (com 90 dias de spin-up antes)
python3 fwi_observado.py --config config.yaml --dias 30

# Um mes fechado + a media mensal do FWI
python3 fwi_observado.py --de 20260701 --ate 20260731 --media-mensal

# Rodada continua: retoma do estado salvo e grava o novo
python3 fwi_observado.py --de 20260801 --ate 20260805 \
    --estado-inicial estado_fwi.nc --salvar-estado estado_fwi.nc

# Conferir as entradas sem calcular
python3 fwi_observado.py --de 20260701 --ate 20260731 --simular
```

Cada dia gera um FWI.OBS.{data}{hora}.nc com os sete campos (FFMC, DMC, DC, ISI, BUI, FWI, DSR) em data/output/2.2/FWI_OBS/netcdf. As variáveis meteorológicas da ERA5 (0,25°) são interpoladas para a grade do IMERG (0,1°), que é a grade de saída. Dias sem entradas completas são pulados com aviso e o estado é mantido, sem quebrar a série.

Opções: --dias/--semanas/--meses/--de+--ate e --data-final (como no rf_observado.py), --spinup N, --estado-inicial/--salvar-estado, --ffmc0/--dmc0/--dc0 (partida a frio, padrão 85/6/15), --media, --media-mensal, --maximo, --var-agrega (componente agregado, padrão FWI), --produto, --hora e --simular.

Convenção de hora: o sistema canadense usa as condições do meio-dia local — sobre o Brasil (UTC-3), ~15 UTC. O padrão do script é 18 UTC apenas para reaproveitar o banco de ERA5 já baixado para o RF; para seguir a convenção à risca, baixe a ERA5 com prepara_era5.py --hora 15 e rode o FWI com --hora 15.

7.3 Figuras do FWI

O rf_figura.py reconhece os arquivos FWI.* e usa automaticamente as classes do índice (0-5-12-22-35), além de permitir plotar qualquer componente:

```
python3 rf_figura.py data/output/2.2/FWI_OBS/netcdf/FWI.OBS.2026073118.nc
python3 rf_figura.py ../FWI.OBS.2026073118.nc --var DC # seca profunda
python3 rf_figura.py ../FWI.OBS.202607*.nc --painel --colunas 7
```

7.4 O que falta para o FWI previsto

O FWI observado fecha a "análise de fogo contínua" da metodologia. Para o FWI previsto falta apenas o vento nas fontes de previsão: o prepara_gfs.py hoje baixa APCP, TMP e RH — acrescentar UGRD/VGRD a 10 m habilita o FWI de curto prazo; nas fontes sazonais (BESM/Eta) o vento precisa vir no pacote de dados. A condição inicial dos códigos de umidade sai naturalmente do estado salvo pelo fwi_observado.py (--estado-inicial).

8. Saídas

8.1 Produto diário (1 a 5 dias)

Saída	Local
NetCDF por horário (19 arquivos)	data/output/2.2/RF_PREV/netcdf/<modelo>/RF.PREV.YYYYMMDDHH.nc
GeoTIFF por horário	data/output/2.2/RF_PREV/tif/<modelo>/RF.PREV.YYYYMMDDHH.tif
Links para o mapserver (D1-D5, 18 UTC)	dados/mapfiles/tmp/RF.PREV.D<dia>.tif

Saída	Local
Cópias T0-T4 (18 UTC)	.../tif/<modelo>/RF.PREV.T<d>.YYYYMMDD18.tif
Fogograma (todos os horários juntos)	data/output/2.2/fogograma/RF.PREV.<data>00.nc

8.2 Produto semanal (1 a 2 semanas)

Saída	Local
NetCDF (2 arquivos: +7 e +14 dias)	data/output/2.2/RF_PREV_SEMANAL/netcdf/<modelo>/
GeoTIFF T7 e T14	.../RF_PREV_SEMANAL/tif/<modelo>/RF.PREV.T7.tif e RF.PREV.T14.tif

8.3 Formato dos arquivos

O NetCDF contém a variável `rbf(time, lat, lon)` com o RF em $[0, 1]$, arredondado a 2 casas decimais, valor ausente -999 e eixo de tempo na data/hora da previsão. O GeoTIFF está em EPSG:4326, orientado de norte para sul, com nodata -999, compressão LZW e organização em tiles.

Interpretação usual do RF: mínimo (< 0.15), baixo ($0.15-0.40$), médio ($0.40-0.70$), alto ($0.70-0.95$) e crítico (≥ 0.95).

9. Logs e monitoramento

Log	Conteúdo
log/log.<modelo>.<previsão>	Progresso e eventuais erros de cada previsão (um por horário)
log.falta.arquivos.prev.prec.txt (diretório de execução)	Arquivos IMERG esperados e não encontrados
Saída padrão	Datas da rodada, horários processados e tempo total

O código de saída é 0 em caso de sucesso e 1 se alguma previsão não foi gerada (mensagem "PROBLEMA - FALTAM ARQUIVOS EM <dir>"), o que permite monitorar a rodada no cron da mesma forma que antes.

10. Solução de problemas

Sintoma	Causa provável	O que fazer
Número de tempos de precipitação insuficiente: N (esperado 120)	Faltam arquivos IMERG no período de 119 dias	Verifique log.falta.arquivos.prev.prec.txt e reponha os arquivos
PROBLEMA - FALTAM ARQUIVOS EM ...	Uma ou mais previsões falharam	Consulte log/log.<modelo>.<previsão> do horário faltante
FileNotFoundException em arquivo GFS.PREV.*	A rodada do GFS ainda não foi processada para o dia	Aguarde/reprocesse a etapa do GFS e rode novamente

Sintoma	Causa provável	O que fazer
A topografia (...) não tem a mesma grade do mapa de vegetação (...)	Arquivo de topografia ou vegetação trocado	Confira os arquivos em data/input/
Consumo excessivo de memória / máquina lenta	Muitos processos simultâneos em grade de 1 km	Reduza --jobs
Falha no envio (lftp/scp)	Rede ou credenciais	O cálculo não é afetado; reenvie manualmente ou rode novamente
ModuleNotFoundError: rasterio (ou outra biblioteca)	Dependências não instaladas no Python usado	source ativa_riscofogo.sh e instale com python3 -m pip install ... (nunca pip solto)
prepara_imer: erro "resposta HTML" do Earthdata	Autenticação incompleta (app não autorizado, login=e-mail ou token expirado)	A mensagem indica o caso; prefira o token (~/.edl_token) — seção 6.2
Segmentation fault em prepara_imer/prepara_gfs	Versão antiga dos scripts (conversão HDF5/GRIB em threads simultâneas)	Atualize: as versões atuais serializam a conversão com lock
python3 continua sendo o 3.6 do sistema após ativar o conda	Env conda sem Python próprio (env "vazio")	conda install -p <caminho_do_env> -c conda-forge python=3.12 ... ou use o venv

11. Testes de validação

Após instalar ou alterar qualquer coisa, rode:

```
python3 teste_rf.py           # valida o núcleo do cálculo (rf_core)
python3 teste_rf_previsto.py  # valida o script genérico de ponta a ponta
python3 teste_rf_multifonte.py # valida o modo multifonte (Eta 13m, BESM 12h, JSON)
python3 teste_prepara_gfs.py  # valida o preparo do GFS (idx, baldes, NetCDF)
python3 teste_prepara_imer.py # valida o preparo do IMERG (conversão, caminhos)
python3 teste_prepara_era5.py  # valida o preparo da ERA5 (UR de T+Td, conversão)
python3 teste_rf_observado.py  # valida o RF observado, as agregacoes e as figuras
python3 teste_fwi.py           # valida o motor FWI (referencia + xclim) e o FWI observado
python3 teste_era5_horario.py  # valida o horario da ERA5 (fixo/solar) e a escala da UR
```

O primeiro teste cria dados sintéticos em /tmp/teste_rf, executa o cálculo completo e compara com uma implementação de referência fiel ao NCL; a saída esperada termina com "TODOS OS TESTES PASSARAM". O segundo monta uma árvore com a estrutura de diretórios da produção em /tmp/teste_generico e executa o rf_previsto.py real em cinco cenários (lista, intervalo, fallback do GFS, horizonte absoluto e falha controlada).

12. Segurança

O script semanal contém as credenciais do lftp (herdadas do shell script original) nas constantes LFTP_USUARIO e LFTP_SENHA. Recomenda-se movê-las para variáveis de ambiente ou para o arquivo ~/.netrc e restringir a permissão de leitura dos scripts.

13. Suporte

Modelo: Alberto Setzer (alberto.setzer@inpe.br) • Código NCL original: Guilherme Martins (guilherme.martins@inpe.br) • Programa Queimadas: <http://www.inpe.br/queimadas/>